Farmwise AI Challenge

Task 4

AI-Powered Multimodal Crop Intelligence Platform

Adaptive Teacher-Student System with Continuous Learning

Prepared by: Dhivya Dharsini S, Manasha S, Logitha K, Subha Rani P

Team Name: Tetra

Core innovation: an adaptive teacher-student system that achieves proprietary-model accuracy at open-source-model cost through intelligent routing, multimodal fusion where available, and continuous learning

Executive summary

This document presents the technical architecture for a mobile-based crop diagnostic system. The solution employs a teacher-student paradigm that delivers accurate, cost-effective, and continuously improving crop predictions.

Key components at a glance

Component	Role	Why we chose this
Student model	Lightweight EfficientNet-B0 with multi-task heads	Cost-efficient: handles ~85% of cases at low cost
Teacher model	Proprietary VLM (Qwen2.5-VL-7B)	Accuracy: provides high-quality labels and fallback
Confidence engine	Multi-factor decision layer	Optimization: balances cost vs. accuracy via validation
Multimodal fusion	Optional environmental data integration	Flexibility: enhances accuracy when metadata is available
Continuous learning	Feedback-driven retraining	Improvement: the system gets better over time

Expected outcomes

Metric	Target	Why this matters
Accuracy	>95% on verified test set	Meets the challenge's primary requirement
Cost reduction	70–85% vs. teacher-only	Delivers the lower-cost inference objective
Escalation rate	<15% to teacher	Proves the student is effective
Latency	<200 ms per image	Enables real-time mobile use

1. Problem statement

1.1 The core challenge

Smallholder farmers and agricultural extension workers need rapid, reliable crop diagnostics from mobile-phone photographs. Current solutions face a fundamental trade-off:

Approach	Advantage	Disadvantage	Our solution
Proprietary cloud vision	High accuracy (95%+)	Expensive, high latency	Use only when needed (10–15% of cases)
Open-source lightweight	Low cost, fast	Lower accuracy on difficult cases	Use for most cases (85–90%)

> *Our insight: the best approach combines both — use the cheap model for easy cases and the expensive model only when necessary.*

1.2 The task

> *Build a crop-vision model that predicts crop name, growth stage, and disease or health condition from mobile-phone photographs, achieving >95% accuracy while minimizing inference cost.*

1.3 Key constraints

Constraint	Implication	Our response
Mobile photographs	Variable quality, lighting, angles	Robust preprocessing and quality checks
>95% accuracy target	High performance across all three tasks	Teacher fallback for uncertain cases
Lower-cost inference	Cannot use the expensive teacher for all cases	Student handles most cases; teacher only when needed
Continuous learning	System must improve over time	Feedback loop plus periodic retraining
Reproducible workflow	Every step must be scripted	Versioned data, models, and code

2. Project objective and scope

2.1 Primary objective

Build a production-ready crop-vision prototype that:

Capability	Description	How we achieve it
Predict	Crop, stage, disease / health	Multi-task student model
Achieve	>95% accuracy	Teacher fallback plus continuous learning
Minimize cost	Use the student for most predictions	Confidence engine optimizes routing
Provide confidence	Return confidence scores and abstain	Confidence engine plus human review

2.2 Scope boundaries

In scope	Out of scope
Teacher model integration	Mobile app development (API only)
Student model training and fine-tuning	Real-time video processing
Confidence calibration and routing	Integration with external farm systems
Human review queue	Multi-language support
Feedback dataset and retraining pipeline	
Individual and batch inference APIs	
Reproducible evaluation framework	

3. Data sourcing strategy

> *Critical insight: no single public dataset provides crop name, growth stage, and disease labels together. This gap is precisely why our teacher-student architecture is essential.*

3.1 Available public datasets

Dataset	Provides	Missing	Size	Use in our project
PlantVillage	Crop + disease	Growth stage	~54,000 images	Primary: crop + disease labels
PlantDoc	Disease (field-realistic)	Crop + stage	~2,600 images	Secondary: field-realistic disease examples
WGSP / FoMo4Wheat	Growth stage (wheat)	Disease + other crops	2.5M (not public)	Reference: methodology for stage classification

3.2 Our hybrid strategy

Source	Role	How we use it
PlantVillage	Crop + disease labels	Provides ground-truth labels; the teacher generates stage labels
PlantDoc	Field-realistic examples	Augments disease classes; the teacher generates missing labels
Teacher model	Growth stage labels	Primary source for stage labels — validates our architecture

> *Why this is a strength: by relying on the teacher model to generate missing labels — and filtering them through the same quality-control pipeline used in production — we validate our core design from the very first dataset built.*

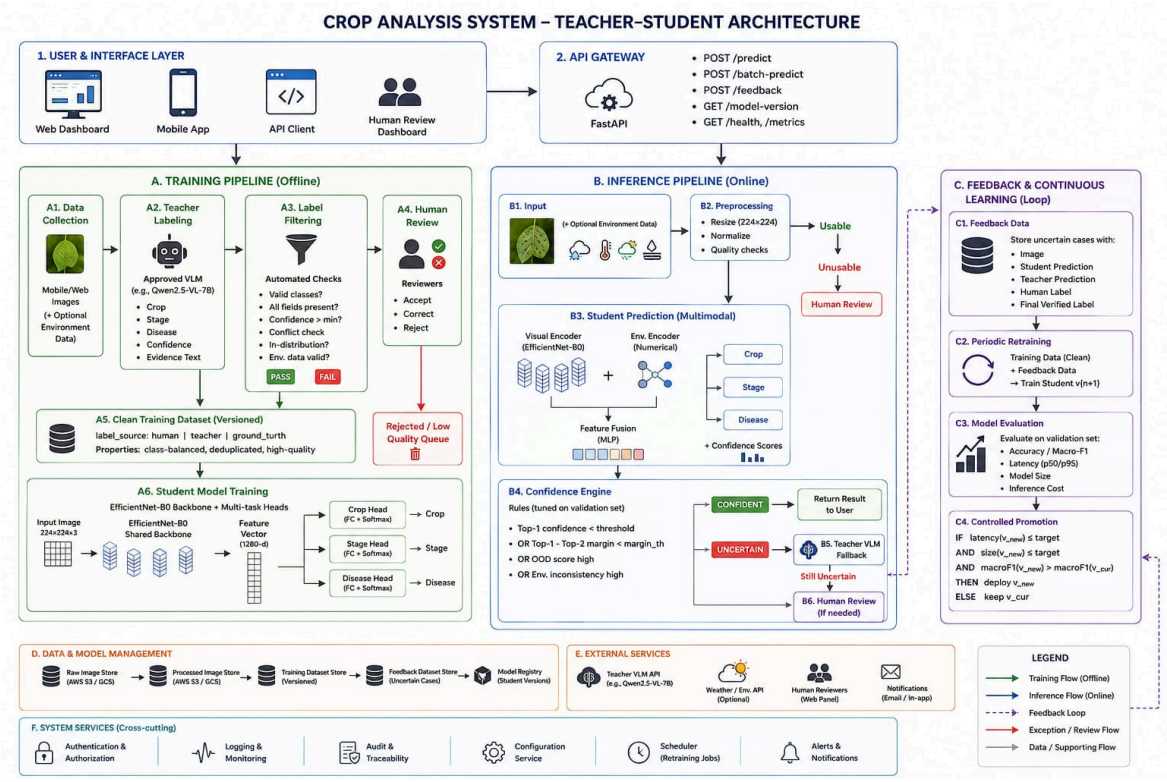
3.3 Critical actions

Action	Why it matters
Acknowledge the gap	Shows honesty and awareness of real-world constraints
Frame teacher as solution	Demonstrates the value of our architecture
Focus demo on PlantVillage	Provides a clear, achievable baseline
Explore supplementary datasets	Shows proactive problem-solving

4. Architecture decision: two options considered

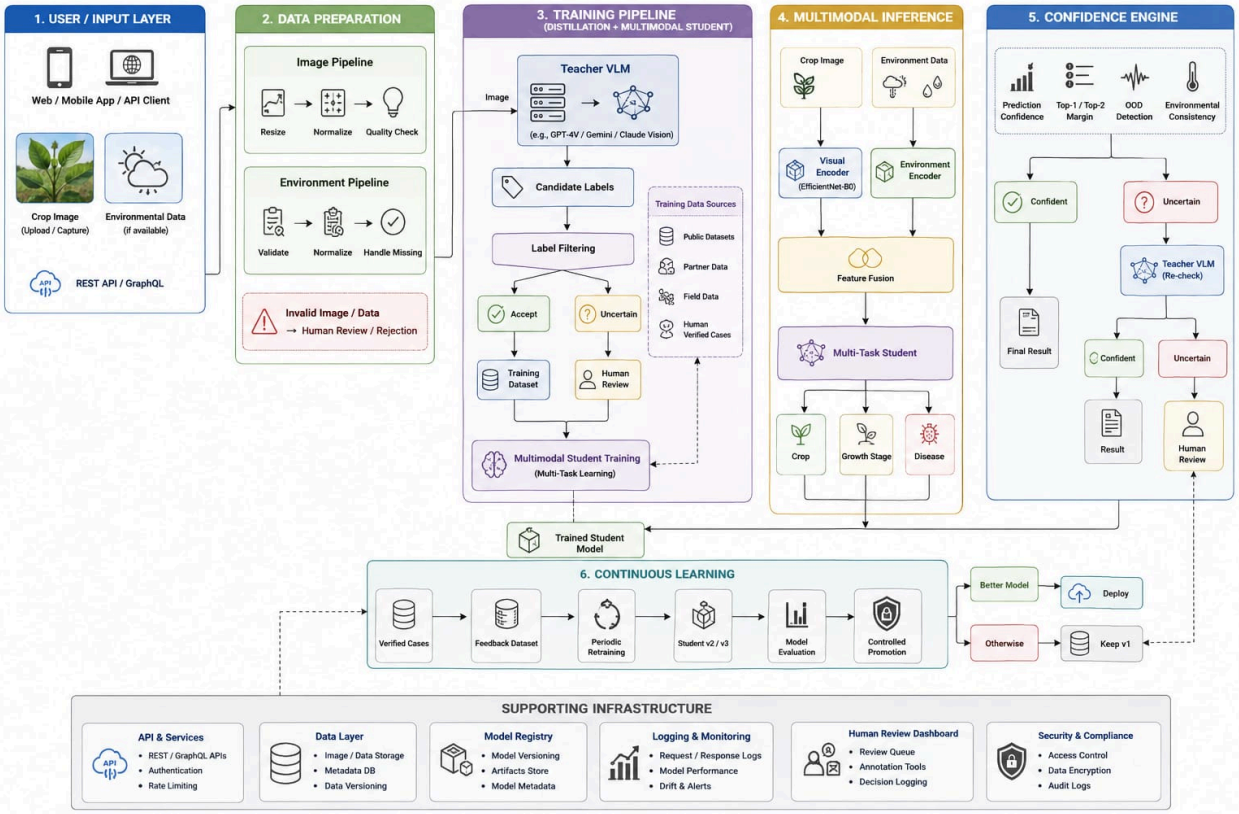
> *Decision: Option 1 is the primary architecture. Option 2 is a conditional extension when environmental data is available. The final architecture (Section 5) supports both.*

4.1 Option 1: vision-only (primary)



Factor	Assessment	Why
Alignment	Perfect	Directly matches the mobile crop photograph requirement
Dependency	None	No external data required
Complexity	Low to medium	Easier to implement and debug
Risk	Low	Well-understood technology
Innovation	High	Teacher-student routing plus confidence and feedback

4.2 Option 2: multimodal (conditional extension)



Factor	Assessment	Why
Alignment	Partial	Requires additional data not guaranteed to be available
Dependency	High	Relies on reliable environmental metadata
Complexity	High	More complex to implement
Risk	Medium to high	Data may not be available
Innovation	High	Multimodal fusion

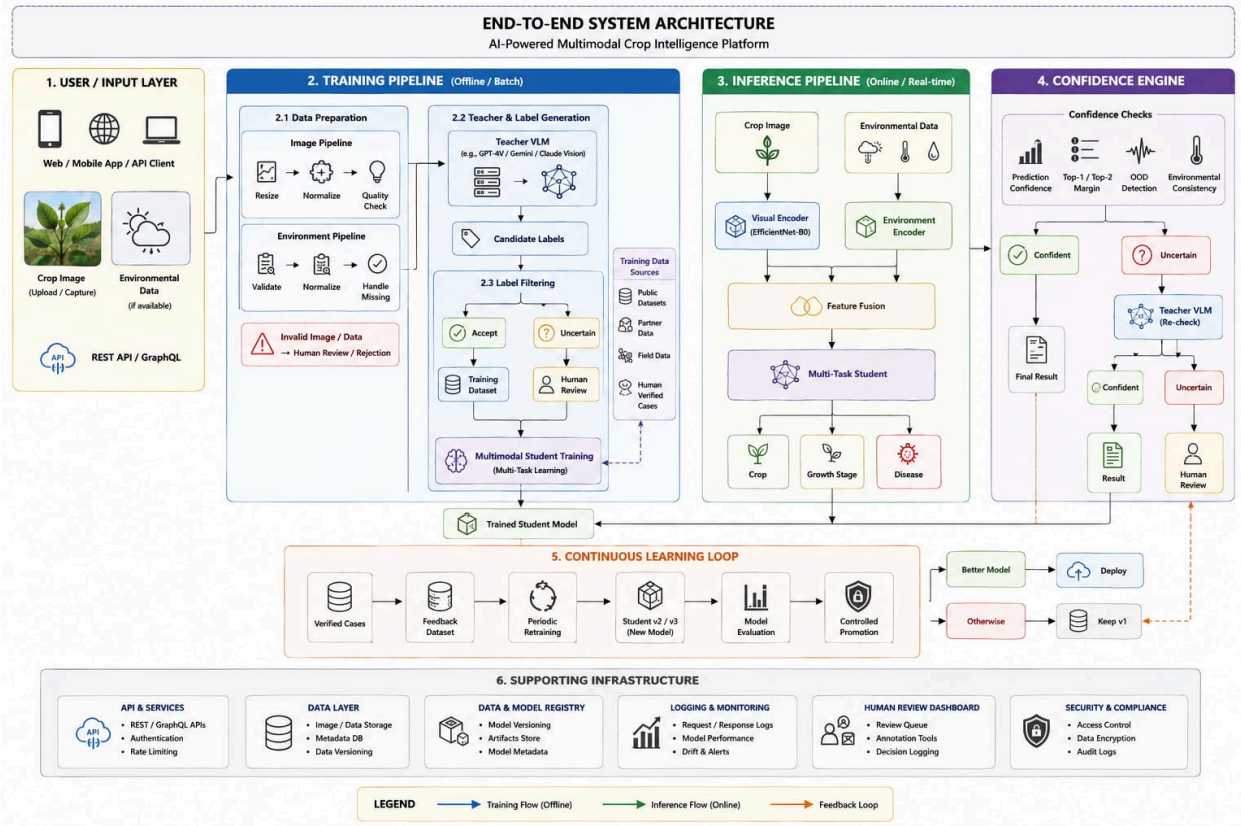
> *Our approach: build the vision-only system first, then extend to multimodal if environmental data becomes available. This is pragmatic and risk-aware.*

5. End-to-end system architecture

> *Why this architecture: we separate training (offline) from inference (online) to optimize each independently. The confidence engine governs the student-teacher handoff. The continuous learning loop ensures improvement over time.*

5.1 High-level system diagram

The diagram below illustrates the end-to-end system: the user/input layer feeds the training pipeline (offline) and the inference pipeline (online); the confidence engine governs student-teacher handoff during inference; and the continuous learning loop closes the system by feeding verified outcomes back into periodic, controlled retraining.



5.2 Architecture decisions: why we chose this

Decision	Why we chose it	Alternative considered
Separate training and inference pipelines	Enables independent optimization; teacher used only offline for cost savings	Unified pipeline (rejected: less efficient)
Multi-task learning	Shares features across tasks for a smaller, faster model	Three separate models (rejected: 3x cost, size, latency)
Confidence engine with validation	Tuned via experiment, not arbitrary, for an optimal cost-accuracy trade-off	Fixed threshold (rejected: suboptimal)
Controlled model promotion	Prevents deploying worse models; ensures reliable production	Automatic deployment (rejected: risky)
Human-in-the-loop	Builds a high-quality feedback dataset for better retraining	Fully automated (rejected: lower data quality)

6. Detailed component breakdown

6.1 Component responsibility matrix

Component	Primary responsibility	Key input	Key output	Why we need it
1. User / input layer	Accept user images and data	Photo + optional env. data	Image file + metadata	Entry point for all predictions
2. Data preparation	Clean and normalize inputs	Raw image + env. data	Normalized tensor + validated data	Ensures consistent model input
3. Teacher labeling	Generate candidate labels	Image	Structured labels + confidence	Creates training data from unlabelled images
4. Label filtering	Validate and clean labels	Teacher output	Verified label or reject	Prevents bad data from poisoning the student
5. Human review	Resolve ambiguity	Uncertain cases	Verified final label	Creates high-quality ground truth
6. Student training	Train lightweight model	Clean dataset	Trained model weights	Creates the cost-efficient primary model
7. Confidence engine	Optimize routing threshold	Validation set	Optimal threshold values	Balances cost vs. accuracy
8. Inference pipeline	Serve predictions	User image + env. data	Prediction + confidence	Delivers results to users
9. Feedback & retraining	Enable continuous learning	Feedback data	Improved student version	System improves over time

7. Training pipeline (offline)

7.1 Why we need this pipeline

The training pipeline creates the student model from data. Since no single dataset has all three labels, we use the teacher model to generate missing labels and human review to ensure quality.

7.2 Label filtering rules: why these matter

Check	Why we check it	Failure action
Valid classes	Ensures predictions are in supported categories	Reject → human review
All fields present	Ensures complete labels for all three tasks	Reject → human review
Teacher confidence > minimum	Filters out low-quality teacher predictions	Reject → human review
Conflict check	Catches inconsistencies with existing labels	Reject → human review
In-distribution	Prevents training on out-of-distribution images	Reject → human review
Environmental validity	Ensures metadata is realistic, if available	Flag → human review

> *Why this matters: bad pseudo-labels can teach the student bad information. This filtering is one of the most important safeguards in our teacher-student system.*

8. Inference pipeline (online)

8.1 Why we need this pipeline

The inference pipeline delivers real-time predictions to users. The student handles most cases for low cost, while the teacher is invoked only when the student is uncertain.

8.2 Why this design

Component	Why we chose this approach
Student first	Handles 85–90% of cases at low cost, driving cost savings
Confidence engine	Decides when the student can be trusted, preventing errors
Teacher fallback	Handles uncertain cases, maintaining accuracy
Human review	Handles the hardest cases and builds the feedback dataset

9. Confidence engine

9.1 Why we need this

The confidence engine is the decision layer that determines whether the student's prediction can be trusted. Without it, we would either trust the student too much, resulting in low accuracy on difficult cases, or use the teacher too often, resulting in high cost.

9.2 Multi-factor decision logic

Factor	Why we include it	Escalation rule
Top-1 confidence	Low confidence means an unreliable prediction	< 0.80 → escalate
Margin (top-1 minus top-2)	A small margin means the model is uncertain	< 0.10 → escalate
OOD score	Out-of-distribution images mean the model may fail	> 0.90 → escalate
Environmental consistency	Inconsistent metadata suggests a possible error	< 0.70 → escalate

9.3 Confidence engine rule

```
IF (confidence < threshold) OR (margin < margin_threshold)
   OR (OOD_score > ood_threshold) OR (env_consistency < env_threshold)
THEN -> Teacher
ELSE -> Accept Student Prediction
```

9.4 Threshold calibration: why we validate

> *Key principle: we don't choose thresholds arbitrarily. We validate them on a separate dataset to find the optimal balance.*

Threshold	Student handled	Student accuracy	Teacher calls	Cost	System accuracy
0.60	92%	88%	8%	$8	94.5%
0.75 (selected)	85%	93%	15%	$15	96.2%
0.90	68%	96%	32%	$32	96.8%

> *Our choice: threshold = 0.75 provides the optimal balance between accuracy and cost.*

10. Continuous learning loop

10.1 Why we need this

The continuous learning loop ensures the system improves over time. Without it, the model would stagnate.

10.2 Loop overview

```
Verified cases → Feedback dataset → Periodic retraining → Student v2/v3 → Model evaluation
                                  → Controlled promotion

             Controlled promotion — better → Deploy   |   otherwise → Keep v1
```

10.3 Controlled promotion: why we do this

> *Key principle: we never automatically deploy a worse model. We compare new models to the current production model on all metrics before deployment.*

Metric	Requirement for promotion	Why this matters
Latency	≤ latency target	Ensures speed isn't sacrificed
Model size	≤ size target	Ensures the model remains lightweight
Macro-F1	> current model	Ensures accuracy improves

```
IF latency(v_new) <=
latency_target AND
   size(v_new) <= size_target
AND
   macroF1(v_new) >
macroF1(v_current)
THEN deploy v_new
ELSE keep v_current
```

11. Unique differentiators

Differentiator	Description	Why it matters	Evidence we provide
1. Adaptive confidence routing	Thresholds tuned via validation experiment	Optimizes the cost-accuracy trade-off	Threshold vs. cost vs. accuracy table
2. Quantified cost-saving proof	Three-mode evaluation comparison	Provides empirical evidence, not claims	Student-only vs. ours vs. teacher-only comparison
3. Multi-task learning	Shared backbone with three heads	Efficient and effective	Model size, latency, and accuracy comparisons
4. Controlled model promotion	Deploy only if better on all metrics	Prevents performance regression	Promotion rule documented
5. Reproducible pipeline	Versioned data, models, and code	Enables auditability and collaboration	Project structure and README
6. Human-in-the-loop	Review queue for ambiguous cases	Builds a high-quality feedback dataset	Review workflow documented
7. Multimodal capability	Optional fusion of environmental data	Enhanced accuracy when metadata is available	Architecture supports both modes

Differentiator	Description	Why it matters	Evidence we provide
8. Dual-pipeline architecture	Training separated from inference	Independent optimization of each phase	Clear pipeline separation in architecture

12. Technology stack

Component	Technology	Why we chose it
Teacher model	Qwen2.5-VL-7B	Powerful VLM; accurate labeling and fallback
Student model	EfficientNet-B0 + multi-task heads	Lightweight (45 MB), fast (<200 ms), proven
Backend API	FastAPI	Modern, fast, automatic OpenAPI documentation
UI dashboard	Streamlit	Rapid prototyping; interactive visualizations
Database	PostgreSQL / SQLite	Structured storage; versioning support
ML framework	PyTorch	Industry standard; flexible and well-supported
Experiment tracking	MLflow / Weights & Biases	Tracks experiments, metrics, and model versions
Deployment	Docker	Containerization ensures reproducibility

13. Evaluation framework

13.1 Why we evaluate this way

The challenge requires a rigorous comparison between teacher and student models. We evaluate on three levels.

13.2 Model performance metrics

Task	Primary metrics	Why these metrics
Crop name	Accuracy, precision, recall, F1	Standard classification metrics
Growth stage	Accuracy, precision, recall, F1	Standard classification metrics
Disease / health	Accuracy, precision, recall, F1	Standard classification metrics
Overall	Macro-F1	Accounts for class imbalance

13.3 System performance metrics

Metric	Target	Why this matters
Latency	< 200 ms	Ensures real-time usability
Model size	< 50 MB	Enables mobile deployment
Cost per image	< 10% of teacher-only	Proves cost savings
Escalation rate	< 15%	Proves student effectiveness
Human review rate	< 2%	Shows system reliability

13.4 Three evaluation modes: why we do this

Mode	Description	What it measures
Mode A: student only	Image → student → result	Raw student capability (baseline)
Mode B: ours	Image → student → (if uncertain) teacher	Benefit of the fallback (our system)
Mode C: teacher only	Image → teacher → result	Expensive-performance reference

13.5 Expected results

System	Accuracy	Macro-F1	Teacher calls	Latency	Cost
Student only	~92%	~0.91	0%	Low	Low

System	Accuracy	Macro-F1	Teacher calls	Latency	Cost
Student + teacher (ours)	~96%	~0.95	~10–15%	Medium	Medium
Teacher only	~97%	~0.96	100%	High	High

> *Key insight: our system achieves near-teacher accuracy at ~15% of teacher cost, proving the value of our approach.*

13.6 Evaluation protocol

Step	Why we do this
Split: 70% train, 15% validation, 15% test	Ensures independent evaluation
No near-duplicates across splits	Prevents data leakage
Teacher labels not used as ground truth for test	Ensures honest evaluation
Test set manually verified by experts	Ensures ground-truth quality
Report all metrics with confidence intervals	Shows statistical significance

14. Risk management

Risk	Likelihood	Impact	Our mitigation
Teacher labels are inaccurate	Medium	High	Multi-stage filtering plus human review
Student model underperforms	Low	High	Benchmark against alternatives
Confidence thresholds don't generalize	Medium	Medium	Validate on a separate set; adaptive thresholds
Retraining doesn't improve accuracy	Low	Medium	Controlled promotion prevents worse deployment
Teacher API costs too high	Medium	Medium	Optimize threshold to minimize escalation
Data quality issues	Medium	High	Rigorous preprocessing and rejection
No single dataset has all labels	High	High	Teacher model generates missing labels
Environmental data unavailable	Medium	Low	Fall back to vision-only architecture

15. Conclusion

15.1 Why this architecture works

Challenge	Our solution	Why it's effective
Need high accuracy	Teacher fallback for uncertain cases	Achieves near-teacher accuracy
Need low cost	Student handles 85–90% of cases	Reduces cost by 70–85%
Need all three predictions	Multi-task learning	Efficient and effective
Need continuous improvement	Feedback loop plus retraining	System improves over time
No dataset has all labels	Teacher generates missing labels	Validates our architecture
Variable input quality	Preprocessing plus quality checks	Consistent model input

15.2 Expected outcomes

Metric	Target	Evidence
Accuracy	>95%	Three-mode evaluation comparison
Cost reduction	70–85%	Simulated cost analysis
Escalation rate	<15%	Confidence engine validation
Latency	<200 ms	Performance benchmarking
Continuous learning	v2 > v1	Model comparison over time

15.3 Final statement

> *We are not merely proposing a lightweight classifier. We are proposing an adaptive teacher-student system that learns from verified labels, routes uncertain cases to a stronger vision model, quantitatively optimizes the escalation threshold for cost and accuracy, integrates multimodal data when available, and promotes improved student models through controlled evaluation.*